

Deep-Learning vs Classical Machine Learning for Log Anomaly Detection on System Log Dataset: A SIEM-Style Evaluation

1st Nubah Nazifa Hossain

Dept. of CSE
BRAC University
Dhaka, Bangladesh
nubah.nazifa.hossain@g.bracu.ac.bd

2nd Munzarin Tabassum

Dept. of CSE
BRAC University
Dhaka, Bangladesh
munzarin.tabassum@g.bracu.ac.bd

3rd Zahidul Hasan Zian

Dept. of CSE
BRAC University
Dhaka, Bangladesh
md.zahidul.hasan.zian@g.bracu.ac.bd

4th Sumaiya Hossain

Dept. of CSE
BRAC University
Dhaka, Bangladesh
sumaiya.hossain@g.bracu.ac.bd

5th Hasan Mahmud

Dept. of CSE
BRAC University
Dhaka, Bangladesh
hasan.mahmud.mazumder@g.bracu.ac.bd

6th Annajiat Alim Rasel

Dept. of CSE
BRAC University
Dhaka, Bangladesh
annajiat@bracu.ac.bd

7th Muhammad Abdur Rahman

Dept. of CSE
BRAC University
Dhaka, Bangladesh
abdur.rahman@bracu.ac.bd

Abstract—Log anomaly detection (LAD) is vital for modern computing systems, where the large volume of generated logs makes manual analysis impractical. Deep learning has demonstrated strong capabilities in modelling complex log patterns; however, most existing studies evaluate these approaches using traditional metrics that do not fully capture the operational effectiveness of a model. This limitation is particularly relevant in Security Information and Event Management (SIEM) environments, where high false positive rates increase alert volume, contribute to analyst fatigue, and reduce the effectiveness of security operations. This thesis compares four log anomaly detection models (LSTM, Autoencoder, LSTM Autoencoder and Random Forest), representing three anomaly detection paradigms (sequence-based, reconstruction-based, and frequency-based), using a SIEM-oriented evaluation framework. By identical pre-processing, blockwise split, and class imbalance handling and thresholding strategies, model performance was assessed using precision, recall, F1-score, false positive rate, specificity, alert volume with training time and inference throughput. The results show that anomaly characteristics have a significant impact on model performance, with the exploratory data analysis confirming that anomalies in HDFS_v1 are mainly frequency-based. Random Forest with random oversampling performed the best, while from deep learning the LSTM-AE achieved best anomaly detection under specific settings. The results support the SIEM-centric evaluation and indicate that evaluations further than benchmark metrics are needed to reveal operational costs such as alert volume and false positive rate.

Index Terms—Anomaly, Deep Learning, HDFS, Log, SIEM

I. INTRODUCTION

System logs record runtime events, system states, warnings and errors. Recording logs is vital for monitoring system health, diagnose faults, and detect security risks to the system. With the huge number of logs received by systems, it has become increasingly important to automate these systems, hence the need to incorporate ML/DL models to make the process faster. For most of LAD research it is seen that the models are evaluated with traditional simple metrics such as accuracy, F1-score, on standardized datasets. This approach does not properly reflect operational needs such as keeping alert volume manageable. A model may score well on benchmarks without controlling false alarms, or detecting anomalies properly. SOC teams that collect, and analyze logs using SIEM software tools are prone to excessive alert overload, hence losing trust in those [14]. This paper tunes and evaluates prominent LAD models using SIEM-related metrics that emphasize managing false positives, alert usability, and confusion-matrix-based measures of operational impact. Our contributions are: (i) a SIEM-oriented evaluation framework that augments standard metrics with FPR, specificity, alert volume, and throughput; (ii) a dataset characterization (using Jensen-Shannon distance over event frequencies) that explains why certain models succeed or fail; (iii) a controlled comparison of class-imbalance handling techniques under identical conditions; and (iv) dual-threshold reporting for reconstruction-based models that exposes the recall–precision trade-off that reporting only single-

threshold would hide.

A. Research Statement

The gap between traditional benchmark-based evaluation and evaluation of models with regard to real operational needs, has motivated this study. There is a substantial lack in assessing model performance based on operational usability [5]. In SIEM environments, usability depends heavily on false-positive behavior, to avoid overwhelming analysts due to alert fatigue [14]. A lack of information into false-positive behavior, alert volume, inference throughout, leaves the operational usefulness of these models poorly understood [5], [14].

B. Research Objective

This research evaluates classical and deep learning models for log anomaly detection under a SIEM-style evaluation framework by moving from benchmark-oriented assessment to an operationally grounded one by training and testing the models on a published system log dataset, using a standardized protocol and reporting both traditional and operational metrics. Four models are investigated: LSTM for sequence-based detection, Autoencoder and LSTM Autoencoder for reconstruction-based detection, and Random Forest as a classical supervised model, all evaluated on the HDFS_v1 benchmark dataset. To deal with the severely skewed dataset, each model is further evaluated under two class-imbalance handling conditions, balanced undersampling versus random oversampling for the supervised model, and mixed versus balanced validation thresholds for the reconstruction-based models.

II. RELATED WORK

Research on Log anomaly detection has progressed from sequence-based representation, such as DeepLog's LSTM next-event prediction [1] to works such as LogAnomaly [27], CNN-based local pattern learning models [8], and Transformer architectures like LogBERT [2], HitAnomaly [3], and BERT-Log [29], exploring quantitative-sequential hybrid representations. Recent works have explored parsing-free [7], graph-based [9] approaches, and LLM-based detectors such as LogLLM [15] but these introduce higher computational costs and increased inference time not efficient for real-time SIEM environments. Comparative studies [16] show reconstruction-based models do not perform as well as sequential models on HDFS, while others [4], [6] have recently argued that classical ML, especially Random Forest is competitive with deep learning, with less computational costs. However, it is an identified gap that most works evaluate models solely using traditional benchmark metrics [12], without emphasising on values related to operational needs for security monitoring systems. Hence we report analysis with false positive rate, alert volume, or analyst workload [5], [14], [17]. Furthermore, handling the huge issue of class imbalance with HDFS_v1 dataset is rarely justified empirically. Studies highly lack adequate model comparisons across paradigms under identical preprocessing[11], [13].

III. DATASET

A. Dataset Selection

The dataset used is the HDFS v1 from LogHub [20]. The raw logs span 38.7 hours and more than 11 million lines, broken down into block-level traces using block IDs, each block given a binary ground truth (normal/anomalous) marked by experts [23]. HDFS v1 is selected over HDFS v2 (unlabeled), HDFS v3 (synthetic, fault-injected anomalies) and BGL (message-wise labeling without natural session grouping) [25].

B. Dataset Description

Blocks are grouped. The preprocessed files used are HDFS.log_templates.csv (29 event templates), Event_traces.csv (block-grouped event ID sequences), anomaly_label.csv (block labels), Event_occurrence_matrix.csv (fixed-length event-count vectors), and HDFS.npz (vectorized inputs with labels). The dataset contains 575,061 blocks. 558,223 normal blocks (97.07%) and 16,838 anomalous (2.93%) [22]. The dataset is compact and is a well-established benchmark.

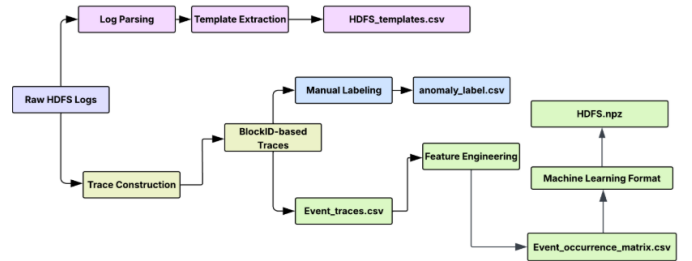


Fig. 1. Preprocessing Pipeline

C. Sequence and Frequency Analysis

Normal blocks have close range of lengths (mean 19.5 events), anomalous blocks vary widely (mean 17.1, standard deviation 12.4), with 25% containing only 4 events. For sequential models, a window size of 20 is justified [1] and highlights a problem: blocks shorter than the window have no sliding windows and LSTM model drops those when evaluating. There are several events that do not occur in normal blocks (E7, E8, E10, E12, E13, E14, E15, E29), and thus any occurrence of these events is a strong anomaly signal. Core execution events (E9, E11, E26) are found in almost all normal blocks, but decrease in anomalous blocks, while error events (E18, E20) increase significantly in anomalies. E22 is present in each block exactly once, not containing any discriminative information.

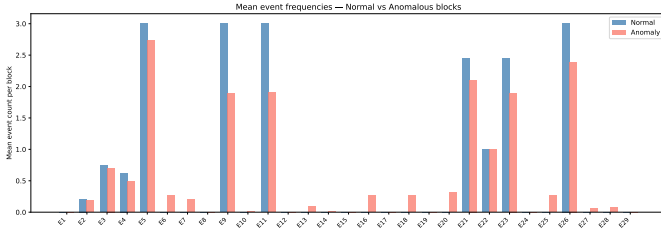


Fig. 2. Event frequencies: normal vs anomalous blocks

D. Jensen-Shannon Distance

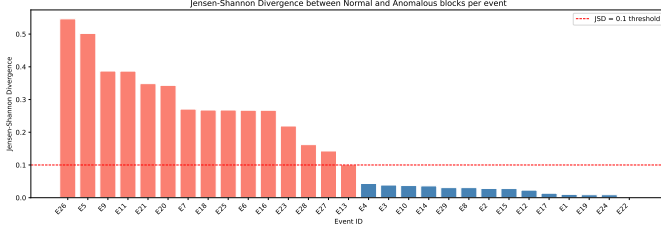


Fig. 3. Jensen-Shannon Distance per event

Jensen-Shannon distance (JSD) per event, for event frequencies showed total of 15 events out of 29 have a value higher than 0.1, and the mean JSD is 0.166. E26 and E5 show the highest divergence (0.546 and 0.502), while E22 is exactly 0.0. This is a direct justification of frequency based detection in HDFS v1 [22] since JSD is calculated based on number of events and not order. This characterization reveals that HDFS v1 anomalies are mostly based on deviations in frequency of events.

IV. METHODOLOGY

A. Preprocessing and Splits

The standardized LogHub preprocessed files are used directly [20]. The blocks are split in ratio of 70/15/15 into train, validation, and test sets to prevent data leakage, using a fixed random seed of 42 [6]. Chronological splitting was avoided due to the lack of reliable timestamps in this preprocessed version. Sequential models use a window size of 20, according to the mean trace length of 19.43 found from the EDA, and with a stride of 1, following standard practice [1], and shorter sequences are padded with the padding tokens masked during training so they aren't mistaken for real events [18], [21]. For the frequency-based models, event-occurrence vectors are max-normalized before being used as input [20].

B. Class Imbalance Handling

Two class imbalance handling techniques are applied according to paradigms and the models are evaluated on those conditions. For the supervised Random Forest, Condition 1 uses balanced undersampling (training on 6,000 normal and 6,000 anomalous blocks) [18], [28] and Condition 2 uses random oversampling (390,000/390,000), reaching 50% minority class proportion, a method deemed effective by

TABLE I
TWO EXPERIMENTAL CONDITIONS PER MODEL TO HANDLE CLASS IMBALANCE

Model	Condition 1	Condition 2
RF	Balanced undersampling (~6k/6k)	Random oversampling (~390k/390k)
AE	Normal-only training + natural validation threshold	Normal-only training + balanced validation threshold
LSTM-AE	Normal-only training + natural validation threshold	Normal-only training + balanced validation threshold
LSTM	Normal-only training, $g = 9$	No second condition; balanced val has no effect since detection rule is fixed at $g = 9$

Guan et. al [15], following the approach of Alsalmi et. al [9]. Synthetic oversampling with SMOTE is excluded. With log data reducing 575,061 sequences to only 26,814 unique patterns, simple duplication achieves the same performance gains without introducing unnatural event sequences [9]. For the semi-supervised reconstruction based models (AE, LSTM-AE), imbalance is not addressed at the training level since they train only on normal data; instead, reconstruction thresholds are calibrated on either a natural [1] or a balanced validation set [24]. The LSTM model is excluded from this comparison as its top- g detection rule is fixed and independent of validation set composition.

C. Models

LSTM (sequence-based): Following DeepLog [1], an embedding layer (dim 32) maps 29 event IDs to dense vectors, two stacked LSTM layers (64 units each), and a softmax output is used to predict the next event. An anomalous block is one where any next event which is predicted is not in the top- g candidates ($g = 9$). One window flagged anomalous deems the entire block an anomaly, to avoid inflating alert counts [1],[19]. Error scored with categorical cross-entropy loss, optimizer Adam, learning rate 0.001 and early stopping patience 3.

Autoencoder (reconstruction-based, frequency): Trained on 29-dimensional normal data event-occurrence frequency vectors with latent dimension of 16 with two encoder and decoder layers. The decoder reconstructs normal frequency distributions [30]. When the reconstruction MSE loss exceeds a threshold calculated on the validation set, a block is flagged anomalous[13]. ReLU hidden activation function, linear output and optimizer Adam, learning rate 0.001. Order is disregarded, only counts are modeled. The model is able to learn unlabeled normal data, an advantage in label-scarce SIEM settings.

LSTM Autoencoder (sequence-based, reconstruction): Encoder LSTM compresses normal sequences, encodes them into a latent vector and decoder LSTM reconstructs those [24]. An anomaly is flagged when the reconstruction error (scored with mean sparse categorical cross-entropy) is above the threshold. This model captures richer patterns of both

event order and co-occurrence, giving a theoretical edge over the plain AE on sequence differing anomalies and over the plain LSTM on frequency differing anomalies, albeit with a higher computational cost [10]. Two thresholds reported are F1-optimized and SIEM-oriented(maximizing recall, minimising FPR).

Random Forest (classical supervised): Trained on labeled blocks as 29-dimensional event-frequency vectors, max-normalized. Predicted class-1 probability is the anomaly score [6]. 200 decision trees, with no max depth set, min samples per leaf 1, random seed 42. The threshold is F1-optimized on the validation set, i.e. the F1-score is maximised at the threshold. Class imbalance is handled with undersampling or oversampling.

D. SIEM-Style Evaluation

Every positive prediction is considered an alert [14]. We compute a block-level confusion matrix and report: accuracy, precision, recall (sensitivity), F1-score, and as operational indicators FPR, Specificity, alert volume = TP + FP, training time. Inference time(latency) and throughput reported. We report ROC-AUC and PR-AUC values. A shared function computes all metrics identically across models. The experiments were coded on Google Colab with 12 GB RAM, using the T4 GPU. Deep models used TensorFlow/Keras and RF used scikit-learn, all loading from the same preprocessed split.

V. RESULTS

This chapter contains the experimental results obtained from evaluating the models- Autoencoder(AE), Random Forest(RF), LSTM and LSTM-AE on the HDFS_v1 benchmark dataset under the SIEM-style evaluation framework defined in Chapter 4 with two class imbalance handling conditions. The chapter reports and compares detection performance for each model individually, reports threshold decisions, shows statistical comparisons, and discusses the implications for SIEM deployment along with key limitations.

A. Quantitative Results

TABLE II
MODEL PERFORMANCE COMPARISON

Model	Condition	Prec.	Recall	F1	FPR	Alert Vol.	Throughput (blk/s)
RF	Undersampling	0.9784	0.9912	0.9848	0.0007	2,542	62,735
RF	Oversampling	0.9976	0.9992	0.9984	0.0001	2,513	160,724
AE	Natural val.	1.0000	0.4552	0.6256	0.0000	1,144	16,780
AE	Balanced val.	1.0000	0.4560	0.6263	0.0000	1,144	11,891
LSTM-AE	Natural val., F1 thr.	0.6473	0.7593	0.6988	0.0124	2,943	24,712
LSTM-AE	Natural val., SIEM thr.	0.3703	0.8900	0.5230	0.0453	6,030	24,712
LSTM-AE	Balanced val., F1 thr.	0.7063	0.6277	0.6647	0.0078	2,230	36,178
LSTM-AE	Balanced val., SIEM thr.	0.4037	0.8876	0.5550	0.0393	5,516	36,178
LSTM	Fixed $g = 9$	1.0000	0.0533	0.1011	0.0000	54	10,052

In all four tables, the Random Forest is the best performing model, with high accuracy, precision, recall, and AUC, low false positive rates, and high throughput. These results indicate that it achieves a high and balanced performance in detecting anomalies while making fewer mistakes, therefore

it outperforms all models across both traditional and SIEM-oriented evaluation metrics. This advantage is also due to the supervised nature of the model. Random Forest learns from both normal and anomaly labels, while the deep learning models were trained on normal blocks only. Though labelled anomalies may not always be present in practice.

B. Final Design Adjustments

This section describes the threshold selection decisions made for reconstruction based models, which are AE and LSTM-AE on the validation set.

Autoencoder (natural validation, F1-optimized): Both conditions gave nearly identical results (1.0 precision, 0.4552 recall, 0.6256 F1, 0.0 FPR), the threshold is 0.005929. The reconstruction errors of normal blocks and anomalous blocks are clearly separated, hence threshold value is not affected by the validation balance condition. Condition 1 was selected for its faster throughput 16,780 vs. 11,890 blocks/s.

LSTM-AE (natural validation, F1-optimized): The results as follows: 0.6988 F1, 0.7593 recall, 0.6473 precision, 0.0124 FPR, 2,943 alerts. The SIEM threshold traded precision for recall with results 0.89 recall, 0.37 precision, alert volume 6,030, over 63% false positives). The F1-optimized threshold is better as it generates roughly one false alarm per seven alerts.

C. Overall Best Results

TABLE III
OVERALL BEST RESULTS PER MODEL

Model	Condition	Prec.	Recall	F1	FPR	Alert Vol.	Throughput (blk/s)
RF	Oversampling	0.9976	0.9992	0.9984	0.0001	2,513	160,724
AE	Natural val.	1.0000	0.4552	0.6256	0.0000	1,144	16,780
LSTM-AE	Natural val., F1 thr.	0.6473	0.7593	0.6988	0.0124	2,943	24,712
LSTM	Fixed $g = 9$	1.0000	0.0533	0.1011	0.0000	54	10,052

Random Forest obtains its best result under oversampling, while the AE and LSTM-AE perform best under natural validation.

D. Cross Model Detection Quality

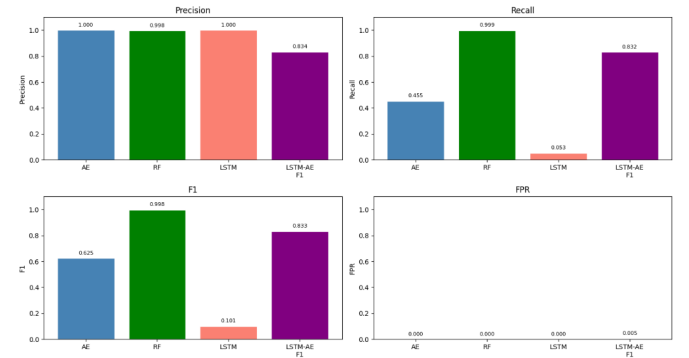


Fig. 4. Cross-model detection quality comparison.

From Fig. 2, RF and AE outperform LSTM-AE and LSTM, in terms of F1 score. F1 scores of Random Forest with

oversampling for class imbalance handling is 0.998, 0.833 for LSTM-AE, 0.6245 is for AE and LSTM only 0.101.

This is consistent with the EDA, where 15 out of 29 event types have a Jensen-Shannon distance ≥ 0.1 , indicating that classes vary in the frequency of events, but not in the sequence of events. HDF5_v1 anomalies are represented as unexpected numbers of events and not essentially as changes in sequence. This is why the models learning event frequencies work, and the LSTM with a sequence has little to work with.

E. Operational Metrics

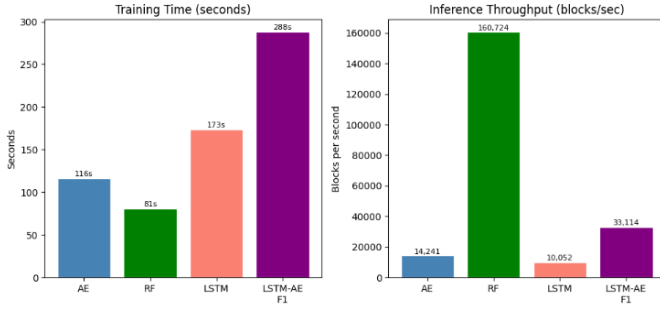


Fig. 5. Operational metric comparison across models.

Random Forest is also the most operationally efficient: 81 s training time, 0.54 s inference time, and 160,724 logs are processed per second, with an alert volume of 2,513 against 2,509 true anomalies. The Autoencoder has throughput 11,900 blocks/second and raises 1,144 alerts. LSTM trains in 173 s processing 10,052 blocks/second, while LSTM-AE trades throughput for its alert burden, which is based on thresholds.

F. Statistical Analysis

TABLE IV
PAIRWISE McNemar's TEST RESULTS COMPARING AE, RF, AND LSTM-AE MODELS

Model Pair	McNemar Stat.	p-value	Interpretation
AE vs RF (Oversampling)	1,369.00	< 0.0001	Statistically significant
AE vs LSTM-AE	1,799.00	< 0.0001	Statistically significant
RF (Oversampling) vs LSTM-AE	112.60	< 0.0001	Statistically significant

For the class imbalance, PR-AUC is used as the threshold-independent measure; RF has 1.0, followed by AE of 0.986 and LSTM-AE has 0.649. McNemar's test was used to compare the three models AE vs RF, AE vs LSTM-AE, RF vs LSTM-AE in pairs, to show the significant differences are not due to sampling error but represent a real model architecture difference. The LSTM was excluded from pairwise testing as it was evaluated on different test set as shorter blocks were not tested, making its error counts non-comparable.

G. Feature Importance

Gini and SHAP analyses agree that a small set of event-count features drives RF's decisions. E20 accounts for 34% of impurity reduction and E26 is 14%, with E27, E5, E11, and

E9 next. This is in line with the JSD analysis, which indicated that frequency features contain the most discriminative signal as E26 and E5 have the greatest distributional divergence.

H. Comparison with Prior Work

RF's oversampling F1 (0.9984) closely matches Ali et al. [6] (0.9978) using frequency-based block features. Our LSTM's low F1 (0.1011) contrasts with DeepLog [1] (0.96) and Ali et al. [6] (0.9853). The AE result (0.626) is greater than Kavlak and Sezen's [16] 0.408, as the inputs are frequency-vector, and the LSTM-AE (0.699) is close to Bekkouche et al.'s [24] 0.737.

I. Discussion

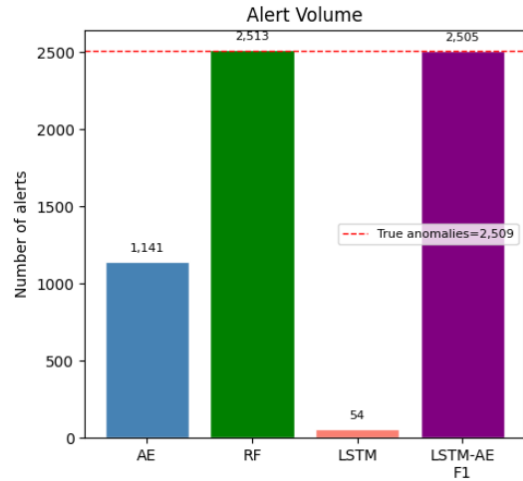


Fig. 6. Alert volume across the models

Random Forest has maximum F1 and recall and minimum FPR with the highest alert/detection ratio (close to 1.00:1) and is recommended for deployment in SIEM for HDF5_v1 style structured logs. At the optimized threshold (F1), LSTM-AE shows the best performance among the deep learning models, in terms of detecting the maximum number of anomalies while keeping the false alarms to a manageable level. LSTM is not recommended and the Autoencoder, although having a perfect precision, fails to detect more than half the anomalies. When considering analyst workload as total alert volume, RF raises 2,513 alerts for 2,509 anomalies (ideal in real systems) and the LSTM-AE produces 2,505 alerts, so that nearly every alert corresponds to a true anomaly. In contrast, the AE raises 1,141 genuine alerts, but is unable to detect half the anomalies, which is a detection issue rather than alert fatigue, the LSTM raises only 54 alerts, missing more than 95% of anomalies. By relating evaluation metrics to alert volume, analyst workload and how well normal blocks were successfully ignored, this analysis reveals operational usefulness that only standard benchmark reporting would miss [26].

VI. CONCLUSION

The performances of four LAD models AE, LSTM, LSTM-AE, and RF were evaluated on HDFS_v1 under a SIEM-like framework that extends the standard metrics with FPR, specificity, alert volume and throughput. The key takeaway is that anomaly structure, rather than the complexity of the model, changes performance: the frequency-based anomalies dominate in HDFS_v1, and the frequency-oriented RF and AE outperform the sequence-based LSTM and LSTM-AE, with RF the best.

A. Limitations

Limitations are for a single dataset scope and offline batch evaluation without streaming. In particular, the dependence of RF on labeled anomalies and concept drift.

B. Future Work

Future work aims to use different dataset like BGL for external validation, extend the study to transformer- and LLM-based models, and to perform a streaming evaluation with drift simulation and label-free threshold calibration, which are more representative of real SIEM deployment.

REFERENCES

- [1] M. Du, F. Li, G. Zheng, and V. Srikumar, "DeepLog: Anomaly Detection and Diagnosis from System Logs through Deep Learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Security (CCS)*, 2017, pp. 1285–1298, doi: 10.1145/3133956.3134015.
- [2] H. Guo, S. Yuan, and X. Wu, "LogBERT: Log Anomaly Detection via BERT," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, Shenzhen, China, 2021, pp. 1–8, doi: 10.1109/IJCNN52387.2021.9534113.
- [3] S. Huang et al., "HitAnomaly: Hierarchical Transformers for Anomaly Detection in System Log," *IEEE Trans. Netw. Serv. Manag.*, vol. 17, no. 4, pp. 2064–2076, Dec. 2020, doi: 10.1109/TNSM.2020.3034647.
- [4] B. Yu et al., "Deep Learning or Classical Machine Learning? An Empirical Study on Log-Based Anomaly Detection," in *Proc. IEEE/ACM Int. Conf. Softw. Eng. (ICSE)*, Lisbon, Portugal, 2024, pp. 403–415, doi: 10.1145/3597503.3623308.
- [5] R. G. Albert, "System Logs Anomaly Detection. Are we on the right path?" *Applied Artificial Intelligence*, vol. 39, no. 1, 2025, doi: 10.1080/08839514.2024.2440692.
- [6] S. Ali, C. Boufaied, D. Bianculli et al., "A comprehensive study of machine learning techniques for log-based anomaly detection," *Empir. Softw. Eng.*, vol. 30, p. 129, 2025, doi: 10.1007/s10664-025-10669-3.
- [7] V.-H. Le and H. Zhang, "Log-based Anomaly Detection Without Log Parsing," in *Proc. IEEE/ACM Int. Conf. Autom. Softw. Eng. (ASE)*, Melbourne, Australia, 2021, pp. 492–504, doi: 10.1109/ASE51524.2021.9678773.
- [8] S. Lu, X. Wei, Y. Li, and L. Wang, "Detecting Anomaly in Big Data System Logs Using Convolutional Neural Network," in *Proc. IEEE DASC/PiCom/DataCom/CyberSciTech*, Athens, Greece, 2018, pp. 151–158, doi: 10.1109/DASC/PiCom/DataCom/CyberSciTech.2018.00037.
- [9] E. Alsalmi, A. Alhuzali, and A. Alhothali, "Log-based anomaly detection of system logs using graph neural network," *Comput. Mater. Continua*, vol. 86, no. 2, pp. 1–20, 2025, doi: 10.32604/cmc.2025.071012.
- [10] A. Aziz and K. Munir, "Anomaly Detection in Logs Using Deep Learning," *IEEE Access*, vol. 12, pp. 176124–176135, 2024, doi: 10.1109/ACCESS.2024.3506332.
- [11] P. Himler, M. Landauer, F. Skopik, and M. Wurzenberger, "Anomaly detection in log-event sequences: A federated deep learning approach and open challenges," *Mach. Learn. Appl.*, vol. 16, p. 100554, 2024, doi: 10.1016/j.mlwa.2024.100554.
- [12] S. He, J. Zhu, P. He, and M. R. Lyu, "Experience Report: System Log Analysis for Anomaly Detection," in *Proc. IEEE ISSRE*, Ottawa, ON, Canada, 2016, pp. 207–218, doi: 10.1109/ISSRE.2016.21.
- [13] M. Landauer, S. Onder, F. Skopik, and M. Wurzenberger, "Deep learning for anomaly detection in log data: A survey," *Mach. Learn. Appl.*, vol. 12, p. 100470, 2023, doi: 10.1016/j.mlwa.2023.100470.
- [14] S. Tariq, M. B. Chhetri, S. Nepal, and C. Paris, "Alert fatigue in security operations centres: Research challenges and opportunities," *ACM Comput. Surv.*, vol. 57, no. 9, Art. no. 224, pp. 1–38, Apr. 2025, doi: 10.1145/3723158.
- [15] W. Guan, J. Cao, S. Qian, J. Gao, and C. Ouyang, "LogLLM: Log-based anomaly detection using large language models," arXiv preprint, arXiv:2411.08561v5 [cs.SE], Apr. 2025, doi: 10.48550/arXiv.2411.08561.
- [16] Y. Kavlak and A. Sezen, "A comparative study of deep learning models for log anomaly detection using the HDFS dataset," in *Proc. 5th Int. Conf. Informatics Softw. Eng. (IISEC)*, Ankara, Türkiye, Feb. 2026, pp. 306–309, doi: 10.1109/IISEC69317.2026.11418409.
- [17] M. Khayat et al., "Empowering security operation center with artificial intelligence and machine learning—A systematic literature review," *IEEE Access*, vol. 13, pp. 1–23, Jan. 2025, doi: 10.1109/ACCESS.2025.3532951.
- [18] X. Li, P. Chen, L. Jing, Z. He, and G. Yu, "SwissLog: Robust and Unified Deep Learning Based Log Anomaly Detection for Diverse Faults," in *Proc. IEEE ISSRE*, Coimbra, Portugal, 2020, pp. 92–103, doi: 10.1109/ISSRE5003.2020.00018.
- [19] Z. Zhao, C. Xu, and B. Li, "A LSTM-Based Anomaly Detection Model for Log Analysis," *J. Signal Process. Syst.*, vol. 93, pp. 745–751, 2021, doi: 10.1007/s11265-021-01644-4.
- [20] J. Zhu, S. He, P. He, J. Liu, and M. R. Lyu, "Loghub: A Large Collection of System Log Datasets for AI-Driven Log Analytics," in *Proc. IEEE 34th Int. Symp. Software Reliability Engineering (ISSRE)*, Florence, Italy, 2023, pp. 355–366, doi: 10.1109/ISSRE59848.2023.00071.
- [21] V.-H. Le and H. Zhang, "Log-based anomaly detection with deep learning: How far are we?" in *Proc. Int. Conf. Softw. Eng. (ICSE '22)*, New York, NY, USA: ACM, 2022, pp. 1356–1367, doi: 10.1145/3510003.3510155.
- [22] M. Landauer, F. Skopik, and M. Wurzenberger, "A Critical Review of Common Log Data Sets Used for Evaluation of Sequence-Based Anomaly Detection Techniques," *Proc. ACM Softw. Eng.*, vol. 1, no. FSE, Art. no. 61, pp. 1354–1375, Jul. 2024, doi: 10.1145/3660768.
- [23] W. Xu, L. Huang, A. Fox, D. Patterson, and M. Jordan, "Detecting Large-Scale System Problems by Mining Console Logs," in *Proc. ACM Symp. Operating Systems Principles (SOSP)*, 2009, pp. 117–132, doi: 10.5555/3104322.3104329.
- [24] M. Bekkouche, M. Meski, Y. Khodja, S. M. Benslimane, and E. Tronci, "Log-based anomaly detection using BiLSTM-Autoencoder," in *Proc. Int. Conf. Networking and Advanced Systems (ICNAS)*, El Tarf, Algeria, 2025, pp. 1–6, doi: 10.1109/ICNAS68168.2025.11298040.
- [25] A. Oliner and J. Stearley, "What Supercomputers Say: A Study of Five System Logs," in *Proc. 37th Annu. IEEE/IFIP Int. Conf. Dependable Systems and Networks (DSN)*, Edinburgh, UK, 2007, pp. 575–584, doi: 10.1109/DSN.2007.103.
- [26] F. Jalalvand, M. B. Chhetri, S. Nepal, and C. Paris, "Alert prioritisation in security operations centres: A systematic survey on criteria and methods," *ACM Comput. Surv.*, vol. 57, no. 6, Art. no. 142, pp. 1–39, 2024, doi: 10.1145/3695462.
- [27] W. Meng et al., "LogAnomaly: Unsupervised Detection of Sequential and Quantitative Anomalies in Unstructured Logs," in *Proc. 28th Int. Joint Conf. Artificial Intelligence (IJCAI-19)*, 2019, pp. 4739–4745, doi: 10.24963/ijcai.2019/658.
- [28] X. Zhang et al., "Robust log-based anomaly detection on unstable log data," in *Proc. 2019 27th ACM Joint Meeting on European Software Engineering Conf. and Symp. Foundations of Software Engineering (ESEC/FSE '19)*, Tallinn, Estonia, Aug. 2019, pp. 807–817, doi: 10.1145/3338906.3338931.
- [29] S. Chen and H. Liao, "BERT-Log: Anomaly Detection for System Logs Based on Pre-trained Language Model," *Appl. Artif. Intell.*, vol. 36, no. 1, 2022, doi: 10.1080/08839514.2022.2145642.
- [30] A. Farzad and T. A. Gulliver, "Unsupervised log message anomaly detection," *ICT Express*, vol. 6, no. 3, pp. 229–237, 2020, doi: 10.1016/j.icte.2020.06.003.